perplexity

Continue from where you left off

Picking up exactly where we left off. The plan is approved. The next deliverable is the **AEU-0 package** — five production-ready artifacts handed directly to your coding agents to begin execution.

AEU-0 EXECUTION PACKAGE

LexCore v4.0 — Agent Kickoff Artifacts

These five documents are the operating contracts every agent reads before touching a single line of code. `HordeMaster` *distributes them.* `ContextKeeper` *maintains them.* `CriticAgent` *enforces them.*

ARTIFACT 1 — `AGENTS.md`

The Living Project Manifest

```
# AGENTS.md — LexCore v4.0 Project Manifest
# Maintained by: ContextKeeper
# Read by: ALL agents before every task
# Version: 1.0.0 | Status: AEU-0 ACTIVE

---

## OPERATING RULES (non-negotiable)

1. Read this file completely before starting any task.
2. Never reference another lane's output without a published contract in this file.
3. Self-verify your output before submitting to CriticAgent.
4. Write no hardcoded secrets, credentials, or raw query content anywhere.
5. Every agent output must be typed, linted, and tested before submission.
6. Soft-delete only — never hard-delete legal documents from the database.
7. No raw query text is ever written to any persistent store. Fingerprint only.
8. Every legal claim in LexDraft/LexAnalysis output must cite a source_chunk_id.

---

## PROJECT IDENTITY

- Product: LexCore v4.0
- Purpose: North America's sovereign legal intelligence database
- Region: GCP northamerica-northeast1 (Montréal, Canada)
- Stack: Python 3.12, FastAPI 0.115, PostgreSQL 15 + pgvector,
```

```
         Redis 7, Celery 5, Ollama (mxbai-embed-large-v1),
         MCP Server SDK, Docker, GitHub Actions, Cloud Run
- Repository: lexcore/
- Main branch: main (protected — no direct push)
- Branch strategy: lane/{N}-{lane-name} per lane

---

## CURRENT AEU STATUS

| AEU | Lane              | Status     | Gate Passed | Notes                     |
|-----|-------------------|------------|-------------|---------------------------|
| 0   | Command & Control | ACTIVE     | —           | This document initialised|
| 1   | Foundation        | PENDING    | —           | Awaiting AEU-0 gate       |
| 2   | Backend Core      | PENDING    | —           | Awaiting AEU-1 gate       |
| 3   | Data Pipeline     | PENDING    | —           | Awaiting AEU-2 gate       |
| 4   | Search & Intel    | PENDING    | —           | Awaiting AEU-3 gate       |
| 5   | Quality           | PENDING    | —           | Awaiting AEU-4 gate       |
| 6   | Ops & Docs        | PENDING    | —           | Runs parallel with AEU-5 |
| 7   | Integration Gate  | PENDING    | —           | Final production promote |

---

## LANE CONTRACTS

### LANE 1 — Foundation
Published by: SchemaArchitect (after AEU-1 gate pass)

DATABASE CONTRACT:
- Schema version: [populated after AEU-1]
- Migration head: [alembic revision ID after AEU-1]
- RLS: ENABLED on audit_log, monitor_rules, conflict_checks, billing_events
- Key tables: tenants, jurisdictions, legal_documents, legal_chunks,
              legal_citations, legal_updates, monitor_rules,
              conflict_checks, billing_events, audit_log,
              query_cache, ingest_jobs, eval_runs, agent_manifest
- Vector dimensions: 1024 (mxbai-embed-large-v1)
- Soft delete: is_deleted BOOLEAN on legal_documents (never hard delete)
- Immutable tables: legal_updates, audit_log (append only, no UPDATE/DELETE)
Status: PENDING

DOCKER CONTRACT:
- Services: postgres, redis, ollama, celery-beat, celery-worker, api
- Startup order: postgres → redis → ollama → celery-beat → celery-worker → api
- Health check port: 8000/health
- Dev port: 8000
Status: PENDING

ENV CONTRACT:
Required variables (see .env.example for documentation):
  DATABASE_URL, REDIS_URL, OLLAMA_URL, GCS_BUCKET_NAME,
  GCS_REGION, SECRET_KEY, STRIPE_SECRET_KEY, STRIPE_METER_ID,
  ENVIRONMENT, LOG_LEVEL, ALLOWED_ORIGINS
Status: PENDING

CICD CONTRACT:
```

```
- CI: runs on every PR — lint, type check, test suite, eval gate
- Deploy canary: 10% traffic, 48h hold, auto-promote on gate pass
- Rollback: one command, < 60 seconds
- Branch protection: PR required, CI must pass, no force push
Status: PENDING

---

### LANE 2 — Backend Core
Published by: APIBuilder + MCPServerAgent (after AEU-2 gate pass)

API CONTRACT:
- Base URL: /api/v1/
- Auth: API key in Authorization header (Bearer {key})
- Tenant context: set via RLS after auth (SET LOCAL app.tenant_id)
- Error format: {error: string, code: string, detail: string}
- All responses include: disclaimer field (legal advice wall)
Status: PENDING

MCP TOOL CONTRACT:
- Tools: get_capabilities, search_legal, research_task,
         get_document, get_citations, check_updates,
         jurisdiction_summary, conflict_check
- Protocol: MCP Server SDK compliant
- Streaming: research_task supports WebSocket streaming
- All tool responses include: disclaimer, source_chunk_ids[] where applicable
Status: PENDING

PRIVACY CONTRACT:
- PII scanner: active on all inbound query payloads
- Raw query storage: PROHIBITED — fingerprint (SHA256) only
- Audit log fields: tenant_id, agent_id, tool_called,
                    query_fingerprint, result_doc_ids, latency_ms, cache_hit
- Privacy header: X-LexCore-Privacy-Policy on first connection
Status: PENDING

BILLING CONTRACT:
- Solo tier: hard cap 10,000 queries/month (429 at limit)
- Solo tier: soft warning at 8,000 queries/month
- Developer tier: Stripe Meters API per-query reporting
- All tiers: billing_events table updated async per query
Status: PENDING

---

### LANE 3 — Data Pipeline
Published by: IngestArchitect (after IngestArchitect completes)

INGEST CONTRACT:
- Task queue: Celery with Redis broker
- Queues: ingest_source, chunk_document, embed_chunk, dead_letter
- Retry policy: 3 attempts, exponential backoff (5m → 25m → 125m)
- Dead letter: status='dead_letter' in ingest_jobs + SRE alert
- License gate: SourceAdapter MUST check LEGAL_DATA_SOURCES.md
                and refuse to ingest any source without CLO clearance
- Quality gate: reject chunks with quality_score < 0.7
```

```
- Archive: every raw document archived to GCS before processing
- Bilingual: EN + FR parallel chunks where applicable
Status: PENDING

CHUNK CONTRACT:
- Max tokens: 512 per chunk
- Overlap: 50 tokens between adjacent chunks
- Section path: required on all chunks (breadcrumb string)
- Embedding dimensions: 1024 (mxbai-embed-large-v1)
- Model version tag: required on all embeddings
Status: PENDING

---

### LANE 4 — Search & Intelligence
Published by: HybridSearchEngineer (after HybridSearch completes)

SEARCH CONTRACT:
- Hybrid formula: (semantic_weight × cosine_similarity)
                + (keyword_weight × ts_rank)
- Default weights: semantic=0.7, keyword=0.3 (configurable per request)
- Quality filter: quality_score >= 0.7 on all chunks
- Multi-language: 'en', 'fr', 'both' supported
- Cache: all results cached with SHA256 fingerprint key
- All results include: data_freshness per jurisdiction
Status: PENDING

RESEARCH TASK CONTRACT:
- Input: natural language question + optional jurisdiction/body_of_law
- Decomposition: 3-5 sub-queries generated from question
- Execution: parallel asyncio search across all sub-queries
- Output: structured research report with:
    primary_sources[], supporting_sources[], citation_chain{},
    jurisdiction_gaps[], confidence_score, data_freshness{},
    disclaimer, source_chunk_ids[]
- Citation hallucination: ZERO TOLERANCE
Status: PENDING

---

### LANE 5 — Quality
Published by: TestWriter + EvalRunner (after AEU-5 gate pass)

QUALITY CONTRACT:
- Test coverage: > 85% required (pytest --cov)
- Type checking: mypy --strict must pass (zero errors)
- Linting: black + isort auto-fixed, flake8 zero violations
- Golden set: 50 cases in tests/golden_set.json
- Precision@5: >= 0.85 (blocks deploy if < 0.80)
- Recall@10: >= 0.80 (blocks deploy if < 0.75)
- NDCG@10: >= 0.78 (blocks deploy if < 0.72)
- Citation accuracy: 100% (blocks deploy at any violation)
- Citation hallucination rate: 0.000 (blocks deploy at any violation)
- Data freshness: 100% (blocks deploy if < 95%)
- Regression: any metric drop > 5% from baseline blocks deploy
Status: PENDING
```

LANE 6 — Ops, Integrations & Docs
Published by: ObservabilityAgent + SREAgent (after AEU-6 gate pass)

OBSERVABILITY CONTRACT:
- Metrics: Prometheus at /metrics
- Logs: structured JSON, correlation IDs, NO raw query content
- Alerts: error_rate > 0.5%, P95 search_legal > 400ms,
          P95 research_task > 3s, dead_letter_depth > 50
- Dashboards: Grafana — request rate, error rate, P95 latency, ingest jobs/hr
Status: PENDING

SRE CONTRACT:
- /health: liveness (fast — process alive only)
- /ready: readiness (checks postgres, redis, ollama, gcs)
- SLA: 99.9% uptime on MCP gateway
- Rollback: one command < 60s
- Runbooks: INC-001 through INC-005 in incidents/ directory
Status: PENDING

INTEGRATION CONTRACT:
- LexConnect-Clio: OAuth2, matter context read, research result write
- LexConnect-NetDocuments: ndConnect, document library write
- LexConnect-iManage: Work 10 REST API, matter folder write
Status: PENDING

COMPLETED OUTPUTS
[Populated by ContextKeeper as each agent completes]

Agent	Output	Status	CriticAgent Approved
HordeMaster	AGENTS.md v1.0.0	COMPLETE	YES
ContextKeeper	Manifest initialized	COMPLETE	YES

OPEN BLOCKERS
[Updated by ContextKeeper as blockers are identified or resolved]

ID	Blocker	Owner	Status
B-1	CLO clearance: CanLII API	CLO	OPEN
B-2	CLO clearance: Justice Laws CA	CLO	OPEN
B-3	Legal SME: golden set annotations	Legal SME	PENDING
B-4	Stripe account: meter ID creation	Product Ops	OPEN
B-5	GCS bucket: northamerica-northeast1	DevOps	OPEN
B-6	Ollama: mxbai-embed-large-v1 pull	Data Engineer	OPEN

ARTIFACT 2 — `LEGAL_DATA_SOURCES.md`

CLO Clearance Registry

```
# LEGAL_DATA_SOURCES.md — LexCore v4.0
# Owner: CLO / Product Counsel
# Read by: ALL SourceAdapter agents before ingesting any source
# Rule: SourceAdapter agents MUST check CLO_CLEARED status.
#       Any source showing PENDING or BLOCKED must not be ingested.
#       Violation of this rule is a critical production failure.

---

## CLEARANCE STATUS KEY

✅ CLEARED    — CLO approved, safe to ingest
⚠️ PENDING    — Under CLO review, DO NOT INGEST
❌ BLOCKED    — CLO has denied clearance, NEVER INGEST
🔲 NEGOTIATE  — Active partnership/licensing negotiation in progress

---

## PHASE 1 SOURCES (Canadian Wedge)

### CA-FED: Justice Laws Canada
- URL: https://laws-lois.justice.gc.ca
- License type: Crown Copyright (Government of Canada)
- Language: Bilingual (EN + FR)
- AI Training Use: Permitted for non-commercial research reproduction
  under Reproduction of Federal Law Order SI/97-5. Commercial AI
  training requires explicit licence — CLO must confirm scope.
- Data residency: GCS northamerica-northeast1
- CLO STATUS: ⚠️ PENDING
- Required action: CLO to confirm that Reproduction of Federal Law
  Order SI/97-5 covers automated AI training ingestion for a
  commercial platform, or negotiate explicit licence with Justice Canada.
- CLO sign-off date: [NOT YET SIGNED]
- Signed by: [PENDING]

### CA-FED + CA-ON + CA-BC + CA-AB: CanLII
- URL: https://www.canlii.org
- License type: ToS-restricted (CanLII Licence Agreement)
- Language: Bilingual where applicable
- AI Training Use: NOT PERMITTED under standard ToS.
  Requires express written permission or API partnership.
- Data residency: GCS northamerica-northeast1
- CLO STATUS: 🔲 NEGOTIATE
- Required action: Formal partnership request to CanLII (non-profit
  owned by Federation of Law Societies of Canada). Proposed terms:
  attribution in all outputs, no redistribution of raw text,
  AI training for research assistance only (not commercial model sale),
  LexCore listed as authorized partner. CLO leading negotiation.
- CLO sign-off date: [PENDING NEGOTIATION OUTCOME]
- Signed by: [PENDING]
- Fallback: If CanLII partnership denied, Phase 1 launches with
```

```
  Justice Laws only for CA sources. CanLII added in Phase 2 post-agreement.

### Common Corpus Legal Commons
- URL: https://commonscrawl.org/legal-commons
- License type: Open License (CC-BY compatible)
- Language: EN + FR + multilingual
- AI Training Use: ✅ Explicitly permitted for AI training
- Data residency: Any (legal corpus data only — no client data)
- CLO STATUS: ✅ CLEARED
- Notes: Verify per-jurisdiction scope on any non-English materials.
         Attribution metadata field required in all ingested documents.
- CLO sign-off date: [TO BE DATED ON CLEARANCE]
- Signed by: [CLO SIGNATURE]

---

## PHASE 2 SOURCES (US Federal + Top 5 States)

### US-FED: eCFR (Electronic Code of Federal Regulations)
- URL: https://www.ecfr.gov/api/
- License type: US Federal Government — Public Domain
  (17 U.S.C. § 105: US government works not subject to copyright)
- Language: EN
- AI Training Use: ✅ Unrestricted — public domain
- Data residency: N/A (public domain — no residency constraint)
- CLO STATUS: ✅ CLEARED
- Notes: Use structured XML API. No scraping required.
         Daily amendment feed available via eCFR versioning API.
- CLO sign-off date: [PRE-CLEARED — US Public Domain]

### US-FED: United States Code (uscode.house.gov)
- URL: https://uscode.house.gov/download/download.shtml
- License type: US Federal Government — Public Domain
- Language: EN
- AI Training Use: ✅ Unrestricted — public domain
- CLO STATUS: ✅ CLEARED
- Notes: Bulk XML download available. No API key required.

### US-FED + US-STATE: CourtListener (Free Law Project)
- URL: https://www.courtlistener.com/api/
- License type: CC-BY 4.0 (PACER mirror — attributed to Free Law Project)
- Language: EN
- AI Training Use: ✅ Permitted with attribution
- CLO STATUS: ✅ CLEARED
- Required action: Attribution metadata field populated on all
  case law documents: source="CourtListener / Free Law Project",
  license="CC-BY 4.0", source_url=[document URL]
- Notes: REST API with bulk data access. Covers federal + all
  state courts via RECAP archive. Rate limit: 5,000 req/day on free tier.
  Request elevated limits via API partnership for Phase 2 launch.

### US-CA: California Statutes (leginfo.legislature.ca.gov)
- URL: https://leginfo.legislature.ca.gov
- License type: California Government Code § 6250 — public records
- Language: EN
- AI Training Use: ⚠ PENDING — public records law permits access
```

  but AI training use is not explicitly addressed.
- CLO STATUS: ⚠ PENDING
- Required action: CLO to review CA Govt Code § 6250 for AI training
  use. Precedent from courts treating government statutes as public
  domain is strong but no explicit CA statutory AI exception exists.
- CLO sign-off date: [PENDING]

US-NY: New York Statutes (legislation.nysenate.gov)
- URL: https://legislation.nysenate.gov/api
- License type: Open Government API
- Language: EN
- AI Training Use: ⚠ PENDING — API available but ToS silent on AI.
- CLO STATUS: ⚠ PENDING

US-TX: Texas Statutes (statutes.capitol.texas.gov)
- License type: Texas Govt Code § 552 — public records
- CLO STATUS: ⚠ PENDING

US-FL: Florida Statutes (leg.state.fl.us)
- License type: Florida Govt § 119 — public records
- CLO STATUS: ⚠ PENDING

US-WA: Washington State (app.leg.wa.gov)
- License type: Washington Public Records Act
- CLO STATUS: ⚠ PENDING

PHASE 3 SOURCES (Full Continental Coverage)

All remaining US states (45 states)
- CLO STATUS: ⚠ PENDING — per-state ToS audit required before
  each state's SourceAdapter is activated.
- Rule: Each state must be individually cleared before ingest.
  A blanket "US public records" clearance is insufficient.

CA-QC: LégisQuébec
- URL: https://www.legisquebec.gouv.qc.ca
- License type: Crown Copyright (Quebec Government)
- Language: Bilingual (FR primary, EN where available)
- Special requirement: Quebec Law 25 PIA required before ingesting
  any Quebec matter data from clients. Legal corpus itself
  (public statutes) does not trigger Law 25 — client query data does.
- CLO STATUS: ⚠ PENDING
- Additional note: French-language content requires French tokenization
  in chunking pipeline (ChunkingEngineer to implement).

CA-ON, CA-BC, CA-AB, CA-MB, CA-SK, CA-NS, CA-NB, CA-PE, CA-NL, CA-NT, CA-NU, CA
- License type: Provincial Crown Copyright (varies)
- CLO STATUS: ⚠ PENDING — each province requires individual review.
  Phase 1 CanLII partnership covers these if agreement is reached.
  Direct provincial scraping is fallback only.

SOURCE ADDITION PROCESS

```
To add a new source after launch:
1. Data Engineer proposes source to CLO with: URL, license type,
   intended use, data volume estimate, and residency requirements.
2. CLO reviews within 5 business days.
3. On clearance: CLO updates this file with signed approval.
4. SourceAdapter agent is then authorized to build adapter.
5. Adapter must read this file and verify CLEARED status before first run.

NO SOURCE MAY BE ADDED TO PRODUCTION WITHOUT THIS PROCESS.
```

ARTIFACT 3 — PIPEDA_COMPLIANCE.md

Privacy Framework

```
# PIPEDA_COMPLIANCE.md — LexCore v4.0
# Owner: Privacy Counsel + CLO
# Audience: Tenants, law societies, enterprise procurement, board review
# Last reviewed: [DATE]

---

## EXECUTIVE SUMMARY

LexCore is designed from the foundation as a privilege-safe, PIPEDA-compliant
legal intelligence platform. This document describes the privacy controls,
data residency architecture, and compliance representations available to
Canadian law firm clients, corporate legal departments, and enterprise tenants.

---

## 1. PIPEDA COMPLIANCE ARCHITECTURE

### Principle 1 — Accountability
LexCore Inc. (Canadian-incorporated entity) is the accountable organisation
for all personal information under its control. A designated Privacy Officer
is responsible for compliance. Contact: privacy@lexcore.ca

### Principle 2 — Identifying Purposes
LexCore processes the following categories of information:
- Tenant account data: name, billing information, contact (for account management)
- Query fingerprints: SHA256 hash of query content (for caching and rate limiting)
- Usage metrics: query counts, latency, tool used, cache hit status
  (for billing, SLA monitoring, and service improvement)

LexCore does NOT process:
- Raw query text (never persisted — processed in memory only)
- Client matter details
- Client names or identifying information
- Attorney-client privileged communications

### Principle 3 — Consent
All tenants execute a Data Processing Agreement (DPA) at account creation.
```

The DPA clearly describes what is collected, why, and retention periods.
No data is used for any purpose beyond service delivery.

Principle 4 — Limiting Collection
Minimal data collection by design:
- No raw query text stored (SHA256 fingerprint only)
- No client matter metadata stored
- Only operational metrics required for billing and SLA

Principle 5 — Limiting Use, Disclosure, and Retention
Query fingerprints: retained 90 days (cache expiry), then auto-purged.
Billing events: retained 7 years (Canadian tax law requirement).
Audit log: retained 2 years (configurable by Enterprise tenants).
Account data: retained for duration of subscription + 1 year post-termination.

LexCore does NOT use any tenant data to train AI models.
LexCore does NOT sell, share, or disclose tenant data to third parties
except as required by Canadian law.

Principle 6 — Accuracy
Tenant account data can be updated at any time via the admin portal.
Legal corpus data is updated per published sync cadence and version-tracked.

Principle 7 — Safeguards

Technical safeguards:
- All data encrypted at rest (AES-256) in GCS and PostgreSQL
- All data in transit encrypted (TLS 1.3)
- Database-level Row Level Security (RLS) — complete tenant isolation
- API key authentication with bcrypt-hashed key storage
- Rate limiting and query cap enforcement per tier
- PrivacyGuard middleware: PII scan on all inbound queries
- No cross-tenant analytics or shared query patterns

Physical safeguards:
- All tenant data hosted in GCP northamerica-northeast1 (Montréal, Canada)
- Physical servers located in Canada
- GCP data centre complies with ISO 27001, SOC 2 Type II, CSA STAR

Organisational safeguards:
- Access to production data restricted to named personnel only
- All access logged and reviewed quarterly
- Incident response team with 24h notification commitment

Principle 8 — Openness
This document is publicly available at https://lexcore.ca/privacy
Privacy policy is machine-readable via X-LexCore-Privacy-Policy response header.

Principle 9 — Individual Access
Tenants may request a complete export of their account data and audit log
by contacting privacy@lexcore.ca. Response within 30 days.

Principle 10 — Challenging Compliance
Privacy concerns may be directed to privacy@lexcore.ca.
If not resolved, tenants may escalate to the Office of the Privacy
Commissioner of Canada at www.priv.gc.ca.

---

2. CLOUD ACT ANALYSIS & SOVEREIGNTY REPRESENTATION

The CLOUD Act Exposure Question
The US Clarifying Lawful Overseas Use of Data (CLOUD) Act (18 U.S.C. § 2703) permits US law enforcement to compel US corporations to produce data stored anywhere in the world, including data in Canadian data centres.

LexCore's Sovereignty Position
LexCore Inc. is a Canadian-incorporated entity with no US parent corporation. GCP (Google LLC) processes data as a data processor under LexCore's Data Processing Agreement and Canadian privacy law.

RESIDUAL RISK: Google LLC is a US corporation. A US court order directed at Google could theoretically compel Google to produce LexCore tenant data. LexCore's DPA with Google prohibits voluntary disclosure and requires Google to notify LexCore of any legal process to the extent permitted by law.

FOR MAXIMUM SOVEREIGNTY: Enterprise tenants may elect the self-hosted tier, in which all infrastructure runs on tenant-owned or tenant-contracted Canadian infrastructure with no US corporate involvement in the data processing chain. This is the only configuration that eliminates CLOUD Act residual risk.

LexCore makes no representation that the standard hosted tier is 100% immune to US legal process. The self-hosted Enterprise tier provides full data sovereignty for firms requiring that standard.

---

3. QUEBEC LAW 25 (Bill 64) COMPLIANCE

Quebec's Act respecting the protection of personal information in the private sector (Law 25, in force since September 2023) imposes additional requirements including Privacy Impact Assessments (PIAs) for:
- Communicating personal information outside Quebec, OR
- Using personal information for profiling or automated decision-making

LexCore's Law 25 Position
LexCore has completed a Privacy Impact Assessment (PIA) for its standard hosted configuration. The PIA is available to Quebec-based tenants on request to confirm adequate protection under Law 25.

For Quebec-based law firm tenants:
- Query data never leaves GCS northamerica-northeast1 (Montréal — in Quebec)
- No automated profiling or decision-making on personal information
- LexCore's PIA is available for review by firm's designated Privacy Officer
- DPA includes Law 25-specific terms for Quebec entities

PIA reference number: [ASSIGNED ON COMPLETION]
PIA completion date: [DATE]
Reviewed by: [Privacy Counsel name]

---

4. ATTORNEY-CLIENT PRIVILEGE ARCHITECTURE

The Privilege Risk
Courts in multiple US and Canadian jurisdictions have found that
transmitting privileged communications to third-party AI platforms
may constitute a waiver of privilege if the platform's ToS permits
use of data for model training or other purposes.

LexCore's Privilege-Safe Design
LexCore is designed so that attorney-client privileged matter
content NEVER needs to enter the LexCore system:

1. LexCore searches a publicly available legal corpus only.
   No client documents, emails, contracts, or matter notes are
   ingested or processed by LexCore at any point.

2. Query text is processed in memory and never persisted.
   No query content can be compelled from LexCore because
   LexCore does not store it.

3. Query fingerprints (SHA256 hash) are irreversible.
   Even if compelled, the hash cannot be used to reconstruct
   the original query text.

4. LexCore's Terms of Service explicitly prohibit:
   - Using LexCore output as legal advice to clients
   - Inputting privileged matter content into LexCore queries
   - Relying on LexCore output without independent attorney verification

5. All LexCore outputs carry a mandatory disclaimer at the API layer:
   "LexCore outputs are legal research for review by licensed counsel.
   They do not constitute legal advice, a legal opinion, or a
   solicitor-client communication."

---

5. DATA PROCESSING AGREEMENT (DPA) SUMMARY

All tenants execute a DPA at account creation covering:
- Data categories processed (see Principle 2 above)
- Processing purposes (service delivery only)
- Sub-processors: GCP (infrastructure), Stripe (billing)
- Sub-processor data residency: GCP northamerica-northeast1 (CA),
  Stripe (billing data — US, not subject to PIPEDA as financial data)
- Retention periods (see Principle 5 above)
- Breach notification: within 72 hours of discovery
- Audit rights: Enterprise tenants may request annual compliance audit
- DPA governing law: Ontario, Canada

Full DPA available at: https://lexcore.ca/dpa
Quebec supplement available at: https://lexcore.ca/dpa-quebec

---

6. INCIDENT RESPONSE — PRIVACY

```
Privacy breach response follows INC-005 (see incidents/INC-005.md):
- Immediate escalation to Privacy Officer + CLO
- Assessment within 4 hours
- OPC notification within 72 hours if breach poses real risk of harm
- Affected tenant notification without undue delay
- Post-incident review and remediation within 30 days
```

ARTIFACT 4 — `GROUNDING_CONTRACT.md`

Citation Grounding Contract for Generative Agents

````
# GROUNDING_CONTRACT.md — LexCore v4.0
# Owner: Applied AI Engineer + QA Lead
# Enforced by: MCP Gateway + EvalRunner
# Applies to: LexDraft agent, LexAnalysis agent
# Non-negotiable — no exceptions, no overrides, no configuration flags

---

## THE FUNDAMENTAL RULE

Every sentence in any LexDraft or LexAnalysis output that makes a legal
claim, references a legal standard, cites a legal principle, or asserts
a legal fact MUST include a source_chunk_id referencing a chunk that
exists in the LexCore database.

An unsupported legal claim in a legal research platform is not a quality
issue. It is a professional liability event for every attorney who
relies on it.

Citation hallucination rate target: 0.000
Any deployment with a single verified citation hallucination is blocked.

---

## OUTPUT SCHEMA CONTRACT

All LexDraft and LexAnalysis outputs MUST conform to this structure.
The MCP gateway enforces this schema before delivering output to client.
Any output that fails schema validation is rejected — not delivered.

```json
{
  "output_type": "draft | analysis",
  "label": "Draft for attorney review — not a final document",
  "disclaimer": "LexCore outputs are legal research for review by
                 licensed counsel. They do not constitute legal advice,
                 a legal opinion, or a solicitor-client communication.
                 All outputs must be independently verified by a
                 qualified legal professional before reliance.",
  "content": {
    "sections": [
      {
````
````

```
        "heading": "string",
        "paragraphs": [
          {
            "text": "string — the paragraph text",
            "claims": [
              {
                "claim_text": "string — the specific legal claim within this paragrap
                "source_chunk_id": "uuid — REQUIRED for every legal claim",
                "source_document_id": "uuid",
                "source_citation": "string — official citation of source document",
                "source_section": "string — section path within source document",
                "confidence": "float — 0.0 to 1.0"
              }
            ]
          }
        ]
      }
    ]
  },
  "all_source_chunk_ids": ["uuid array — all chunks referenced in output"],
  "all_source_citations": ["string array — all official citations referenced"],
  "jurisdiction_scope": ["string array — jurisdictions covered"],
  "body_of_law": "string",
  "generated_at": "ISO timestamp",
  "lexcore_version": "string"
}
```

WHAT COUNTS AS A LEGAL CLAIM

A legal claim requires a source_chunk_id if the sentence:

- States that a law, regulation, or statute requires, permits, or prohibits something
- Cites a case name, citation, or legal precedent
- Asserts that a legal standard, test, or threshold applies
- Makes a statement about rights, obligations, liabilities, or penalties
- References a jurisdiction-specific legal rule or procedure
- Quotes or paraphrases statutory or regulatory language

A legal claim does NOT require a source_chunk_id if the sentence:

- Transitions between sections ("The following analysis considers...")
- States the document's purpose ("This draft agreement governs...")
- Provides general formatting or structural language

ENFORCEMENT LAYERS

Layer 1 — Prompt Enforcement

The LexDraft and LexAnalysis agent prompts explicitly require structured output conforming to this contract. The prompt instructs the LLM:

- To search for source_chunk_ids using search_legal BEFORE writing claims
- To only include claims it can directly ground to a retrieved chunk
- To omit a claim entirely rather than include it without grounding
- To flag any area where it could not find supporting source material in a "coverage_gaps" field rather than inventing support

Layer 2 — Gateway Schema Validation

The MCP gateway validates every LexDraft/LexAnalysis output against the schema above before delivery. Any output with:

- A missing source_chunk_id on a legal claim → REJECTED (not delivered)
- A source_chunk_id that does not exist in the database → REJECTED
- A missing label or disclaimer → REJECTED
- An empty claims array on a paragraph containing legal content → REJECTED

Layer 3 — EvalRunner Verification

EvalRunner runs citation verification on every LexDraft/LexAnalysis output in the golden set evaluation:

1. Extract all source_chunk_ids from output
2. Verify each chunk_id exists in legal_chunks table
3. Retrieve chunk_text for each referenced chunk
4. Verify that the claim_text is semantically consistent with chunk_text (LLM-as-judge evaluation: "Does this chunk support this claim?")
5. Flag any inconsistency as a citation hallucination
6. citation_hallucination_rate = hallucinated_claims / total_claims
7. Rate > 0.000 → deployment blocked

Layer 4 — Audit Trail

Every grounded output is recorded in audit_log with:

- all_source_chunk_ids[] stored as result_doc_ids
- Agent identity recorded as agent_id

- This creates a permanent, retrievable record of what sources supported every legal output ever delivered

COVERAGE GAP HANDLING

When LexDraft or LexAnalysis cannot find a source chunk to support a required legal claim, the agent MUST:

1. NOT fabricate a citation or make an unsupported claim
2. Include a coverage_gaps section in the output identifying:
   - The jurisdiction where coverage is missing
   - The body of law where coverage is missing
   - The specific legal question that could not be answered
3. Recommend that the attorney conduct additional research in that gap area
4. Flag the output with "coverage_incomplete": true at the top level

Coverage gaps are EXPECTED and ACCEPTABLE.
Fabricated citations are NEVER acceptable.

ENFORCEMENT FAILURE PROTOCOL

If a citation hallucination is detected in production:

1. INC-004 fires immediately (see incidents/INC-004.md)
2. Affected agent suspended until root cause identified
3. Affected tenant notified within 4 hours
4. All outputs from that agent in last 24 hours reviewed
5. Root cause analysis completed
6. Fix deployed with full eval run before agent re-enabled
7. Post-incident review document added to incidents/ directory

```
***

## ARTIFACT 5 — Alembic Migration Scaffold
### Production Schema — Complete Migration Set

```python
# migrations/env.py — Alembic environment configuration
from logging.config import fileConfig
from sqlalchemy import engine_from_config, pool
from alembic import context
from app.models import Base
from app.config import settings
```
```

```
config = context.config
config.set_main_option("sqlalchemy.url", settings.DATABASE_URL.replace(
    "postgresql+asyncpg", "postgresql"  # Alembic needs sync URL
))
fileConfig(config.config_file_name)
target_metadata = Base.metadata

def run_migrations_online():
    connectable = engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.", poolclass=pool.NullPool,
    )
    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata,
                          compare_type=True, compare_server_default=True)
        with context.begin_transaction():
            context.run_migrations()

run_migrations_online()
```

```
# migrations/versions/0001_initial_schema.py
"""Initial LexCore v4.0 schema

Revision ID: 0001_initial_schema
Revises: —
Create Date: 2026-04-29
"""
from alembic import op
import sqlalchemy as sa
from pgvector.sqlalchemy import Vector

revision = '0001_initial_schema'
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:

    # Enable required extensions
    op.execute("CREATE EXTENSION IF NOT EXISTS vector")
    op.execute("CREATE EXTENSION IF NOT EXISTS pgcrypto")
    op.execute("CREATE EXTENSION IF NOT EXISTS pg_trgm")

    # ── TENANTS ──────────────────────────────────────────────
    op.create_table('tenants',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('name', sa.Text(), nullable=False),
        sa.Column('tier', sa.Text(), nullable=False),
        sa.Column('api_key_hash', sa.Text(), unique=True, nullable=False),
        sa.Column('stripe_customer_id', sa.Text()),
        sa.Column('stripe_sub_id', sa.Text()),
        sa.Column('query_limit', sa.Integer()),
        sa.Column('self_hosted', sa.Boolean(), server_default='false'),
```

```
        sa.Column('data_region', sa.Text(), server_default='ca-montreal'),
        sa.Column('pipeda_dpa_signed', sa.Boolean(), server_default='false'),
        sa.Column('law25_pia_completed', sa.Boolean(), server_default='false'),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.Column('is_active', sa.Boolean(), server_default='true'),
        sa.CheckConstraint("tier IN ('solo','firm','enterprise','developer')",
                           name='ck_tenant_tier')
    )

    # ── BILLING EVENTS ──────────────────────────────────────
    op.create_table('billing_events',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('tenant_id', sa.UUID(), sa.ForeignKey('tenants.id'),
                  nullable=False),
        sa.Column('billing_period', sa.Text(), nullable=False),
        sa.Column('query_count', sa.Integer(), server_default='0'),
        sa.Column('tier_at_billing', sa.Text(), nullable=False),
        sa.Column('stripe_meter_event_id', sa.Text()),
        sa.Column('amount_cents', sa.Integer()),
        sa.Column('invoiced_at', sa.TIMESTAMP(timezone=True)),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()'))
    )

    # ── JURISDICTIONS ───────────────────────────────────────
    op.create_table('jurisdictions',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('country', sa.Text(), nullable=False),
        sa.Column('level', sa.Text(), nullable=False),
        sa.Column('name', sa.Text(), nullable=False),
        sa.Column('name_fr', sa.Text()),
        sa.Column('code', sa.Text(), unique=True, nullable=False),
        sa.Column('languages', sa.ARRAY(sa.Text()),
                  server_default="ARRAY['en']::text[]"),
        sa.Column('coverage_status', sa.Text(), server_default='pending'),
        sa.Column('last_full_sync', sa.TIMESTAMP(timezone=True)),
        sa.Column('source_urls', sa.JSON()),
        sa.Column('license_cleared', sa.Boolean(), server_default='false'),
        sa.Column('phase', sa.Integer(), server_default='2'),
        sa.CheckConstraint("country IN ('US','CA')", name='ck_jurisdiction_country'),
        sa.CheckConstraint("level IN ('federal','state','province','territory')",
                           name='ck_jurisdiction_level'),
        sa.CheckConstraint("coverage_status IN ('pending','partial','complete')",
                           name='ck_jurisdiction_coverage')
    )

    # ── LEGAL DOCUMENTS ─────────────────────────────────────
    op.create_table('legal_documents',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('jurisdiction_id', sa.UUID(),
                  sa.ForeignKey('jurisdictions.id'), nullable=False),
        sa.Column('doc_type', sa.Text(), nullable=False),
```

```
        sa.Column('body_of_law', sa.Text(), nullable=False),
        sa.Column('title', sa.Text(), nullable=False),
        sa.Column('title_fr', sa.Text()),
        sa.Column('official_citation', sa.Text(), unique=True),
        sa.Column('source_url', sa.Text(), nullable=False),
        sa.Column('content_hash', sa.Text(), nullable=False),
        sa.Column('raw_text', sa.Text()),
        sa.Column('language', sa.Text(), server_default='en'),
        sa.Column('license_type', sa.Text(), nullable=False),
        sa.Column('data_quality_score', sa.Float()),
        sa.Column('effective_date', sa.Date()),
        sa.Column('repeal_date', sa.Date()),
        sa.Column('version', sa.Integer(), server_default='1'),
        sa.Column('is_current', sa.Boolean(), server_default='true'),
        sa.Column('is_deleted', sa.Boolean(), server_default='false'),
        sa.Column('ingested_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.Column('updated_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.CheckConstraint(
            "doc_type IN ('statute','regulation','case_law','template','analysis')",
            name='ck_doc_type'),
        sa.CheckConstraint(
            "body_of_law IN ('civil','criminal','corporate','labor','tax','ip',"
            "'constitutional','administrative','securities','privacy','family','real
            name='ck_body_of_law'),
        sa.CheckConstraint(
            "license_type IN ('public_domain','cc_by','crown_copyright','licensed','
            name='ck_license_type'),
        sa.CheckConstraint("language IN ('en','fr','bilingual')", name='ck_language')
        sa.CheckConstraint("data_quality_score BETWEEN 0 AND 1", name='ck_quality_sco
    )

    # ── LEGAL CHUNKS ──────────────────────────────────────
    op.create_table('legal_chunks',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('document_id', sa.UUID(),
                  sa.ForeignKey('legal_documents.id', ondelete='CASCADE'),
                  nullable=False),
        sa.Column('chunk_index', sa.Integer(), nullable=False),
        sa.Column('section_path', sa.Text()),
        sa.Column('chunk_text', sa.Text(), nullable=False),
        sa.Column('chunk_text_fr', sa.Text()),
        sa.Column('embedding', Vector(1024)),
        sa.Column('token_count', sa.Integer()),
        sa.Column('quality_score', sa.Float()),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.UniqueConstraint('document_id', 'chunk_index',
                            name='uq_chunk_doc_index'),
        sa.CheckConstraint("quality_score BETWEEN 0 AND 1",
                           name='ck_chunk_quality')
    )

    # ── LEGAL CITATIONS ───────────────────────────────────
```

```
    op.create_table('legal_citations',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('source_doc_id', sa.UUID(),
                  sa.ForeignKey('legal_documents.id'), nullable=False),
        sa.Column('target_doc_id', sa.UUID(),
                  sa.ForeignKey('legal_documents.id'), nullable=False),
        sa.Column('citation_text', sa.Text()),
        sa.Column('citation_type', sa.Text(), nullable=False),
        sa.Column('confidence', sa.Float(), server_default='1.0'),
        sa.UniqueConstraint('source_doc_id', 'target_doc_id', 'citation_type',
                            name='uq_citation'),
        sa.CheckConstraint(
            "citation_type IN ('references','overrules','amends','repeals',"
            "'implements','upholds','distinguishes')",
            name='ck_citation_type')
    )

    # ── LEGAL UPDATES (IMMUTABLE) ──────────────────────────
    op.create_table('legal_updates',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('document_id', sa.UUID(),
                  sa.ForeignKey('legal_documents.id'), nullable=False),
        sa.Column('change_type', sa.Text(), nullable=False),
        sa.Column('previous_hash', sa.Text()),
        sa.Column('new_hash', sa.Text()),
        sa.Column('diff_summary', sa.Text()),
        sa.Column('affected_sections', sa.ARRAY(sa.Text())),
        sa.Column('detected_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.Column('confirmed_by', sa.Text(), server_default='auto'),
        sa.CheckConstraint(
            "change_type IN ('created','amended','repealed','restored','corrected')
            name='ck_change_type')
    )

    # ── MONITOR RULES ──────────────────────────────────────
    op.create_table('monitor_rules',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('tenant_id', sa.UUID(), sa.ForeignKey('tenants.id'),
                  nullable=False),
        sa.Column('rule_name', sa.Text(), nullable=False),
        sa.Column('jurisdiction_codes', sa.ARRAY(sa.Text()), nullable=False),
        sa.Column('bodies_of_law', sa.ARRAY(sa.Text())),
        sa.Column('keywords', sa.ARRAY(sa.Text())),
        sa.Column('alert_channel', sa.Text(), nullable=False),
        sa.Column('alert_endpoint', sa.Text()),
        sa.Column('digest_frequency', sa.Text(), server_default='immediate'),
        sa.Column('is_active', sa.Boolean(), server_default='true'),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.CheckConstraint(
            "alert_channel IN ('email','webhook','slack','in_app')",
            name='ck_alert_channel'),
```

```
        sa.CheckConstraint(
            "digest_frequency IN ('immediate','daily','weekly')",
            name='ck_digest_freq')
    )

    # ── CONFLICT CHECKS ──────────────────────────────────────
    op.create_table('conflict_checks',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('tenant_id', sa.UUID(), sa.ForeignKey('tenants.id'),
                  nullable=False),
        sa.Column('matter_name', sa.Text(), nullable=False),
        sa.Column('party_names', sa.ARRAY(sa.Text()), nullable=False),
        sa.Column('query_match_found', sa.Boolean()),
        sa.Column('checked_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()'))
    )

    # ── AUDIT LOG (IMMUTABLE) ────────────────────────────────
    op.create_table('audit_log',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('tenant_id', sa.UUID(), sa.ForeignKey('tenants.id'),
                  nullable=False),
        sa.Column('agent_id', sa.Text(), nullable=False),
        sa.Column('tool_called', sa.Text(), nullable=False),
        sa.Column('query_fingerprint', sa.Text()),
        sa.Column('result_doc_ids', sa.ARRAY(sa.UUID())),
        sa.Column('result_count', sa.Integer()),
        sa.Column('latency_ms', sa.Integer()),
        sa.Column('cache_hit', sa.Boolean(), server_default='false'),
        sa.Column('called_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()'))
    )

    # ── QUERY CACHE ──────────────────────────────────────────
    op.create_table('query_cache',
        sa.Column('query_fingerprint', sa.Text(), primary_key=True),
        sa.Column('result', sa.JSON(), nullable=False),
        sa.Column('jurisdiction_scope', sa.ARRAY(sa.Text())),
        sa.Column('search_config', sa.JSON()),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.Column('expires_at', sa.TIMESTAMP(timezone=True), nullable=False),
        sa.Column('hit_count', sa.Integer(), server_default='0')
    )

    # ── INGEST JOBS ──────────────────────────────────────────
    op.create_table('ingest_jobs',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('jurisdiction_id', sa.UUID(),
                  sa.ForeignKey('jurisdictions.id'), nullable=False),
        sa.Column('source_url', sa.Text()),
        sa.Column('status', sa.Text(), nullable=False),
        sa.Column('trigger_type', sa.Text(), server_default='scheduled'),
```

```
        sa.Column('documents_processed', sa.Integer(), server_default='0'),
        sa.Column('documents_failed', sa.Integer(), server_default='0'),
        sa.Column('documents_skipped', sa.Integer(), server_default='0'),
        sa.Column('error_detail', sa.Text()),
        sa.Column('started_at', sa.TIMESTAMP(timezone=True)),
        sa.Column('completed_at', sa.TIMESTAMP(timezone=True)),
        sa.Column('created_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.CheckConstraint(
            "status IN ('queued','running','complete','failed','dead_letter','paused
            name='ck_ingest_status'),
        sa.CheckConstraint(
            "trigger_type IN ('scheduled','manual','change_detected','recovery')",
            name='ck_trigger_type')
    )

    # ── EVAL RUNS ──────────────────────────────────────────
    op.create_table('eval_runs',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('run_at', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()')),
        sa.Column('git_sha', sa.Text(), nullable=False),
        sa.Column('triggered_by', sa.Text(), server_default='cicd'),
        sa.Column('precision_at_5', sa.Float()),
        sa.Column('recall_at_10', sa.Float()),
        sa.Column('ndcg_at_10', sa.Float()),
        sa.Column('citation_accuracy', sa.Float()),
        sa.Column('citation_hallucination_rate', sa.Float()),
        sa.Column('total_test_cases', sa.Integer()),
        sa.Column('passed_cases', sa.Integer()),
        sa.Column('gate_passed', sa.Boolean(), nullable=False),
        sa.Column('baseline_delta', sa.JSON()),
        sa.Column('notes', sa.Text())
    )

    # ── AGENT MANIFEST ─────────────────────────────────────
    op.create_table('agent_manifest',
        sa.Column('id', sa.UUID(), server_default=sa.text('gen_random_uuid()'),
                  nullable=False, primary_key=True),
        sa.Column('spec_version', sa.Text(), nullable=False),
        sa.Column('lane_contracts', sa.JSON()),
        sa.Column('completed_outputs', sa.JSON()),
        sa.Column('open_blockers', sa.JSON()),
        sa.Column('last_updated', sa.TIMESTAMP(timezone=True),
                  server_default=sa.text('now()'))
    )

    # ═══ INDEXES ═══════════════════════════════════════════

    # HNSW vector index — primary retrieval engine
    op.execute("""
        CREATE INDEX idx_chunks_embedding
        ON legal_chunks USING hnsw (embedding vector_cosine_ops)
        WITH (m = 16, ef_construction = 128)
    """)
```

```
    # Full-text search indexes — EN and FR
    op.execute("""
        CREATE INDEX idx_chunks_fts_en
        ON legal_chunks USING GIN (to_tsvector('english', chunk_text))
    """)
    op.execute("""
        CREATE INDEX idx_chunks_fts_fr
        ON legal_chunks USING GIN (
            to_tsvector('french', coalesce(chunk_text_fr, ''))
        )
    """)

    # Document lookups
    op.create_index('idx_docs_jurisdiction', 'legal_documents',
                    ['jurisdiction_id'],
                    postgresql_where=sa.text(
                        'is_current = TRUE AND is_deleted = FALSE'))
    op.create_index('idx_docs_body_law', 'legal_documents',
                    ['body_of_law'],
                    postgresql_where=sa.text(
                        'is_current = TRUE AND is_deleted = FALSE'))
    op.create_index('idx_docs_hash', 'legal_documents', ['content_hash'])
    op.create_index('idx_docs_citation', 'legal_documents',
                    ['official_citation'],
                    postgresql_where=sa.text('official_citation IS NOT NULL'))

    # Citation graph traversal
    op.create_index('idx_citations_source', 'legal_citations',
                    ['source_doc_id'])
    op.create_index('idx_citations_target', 'legal_citations',
                    ['target_doc_id'])

    # Audit and monitoring
    op.create_index('idx_audit_tenant_time', 'audit_log',
                    ['tenant_id', sa.text('called_at DESC')])
    op.create_index('idx_updates_doc_time', 'legal_updates',
                    ['document_id', sa.text('detected_at DESC')])
    op.create_index('idx_ingest_status', 'ingest_jobs',
                    ['status', 'jurisdiction_id'])
    op.create_index('idx_billing_tenant_period', 'billing_events',
                    ['tenant_id', 'billing_period'])

    # ═══ ROW LEVEL SECURITY ═══════════════════════════════

    op.execute("ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY")
    op.execute("ALTER TABLE monitor_rules ENABLE ROW LEVEL SECURITY")
    op.execute("ALTER TABLE conflict_checks ENABLE ROW LEVEL SECURITY")
    op.execute("ALTER TABLE billing_events ENABLE ROW LEVEL SECURITY")

    op.execute("""
        CREATE POLICY tenant_audit_isolation ON audit_log
        USING (tenant_id = current_setting('app.tenant_id')::UUID)
    """)
    op.execute("""
        CREATE POLICY tenant_monitor_isolation ON monitor_rules
```

```
        USING (tenant_id = current_setting('app.tenant_id')::UUID)
    """)
    op.execute("""
        CREATE POLICY tenant_conflict_isolation ON conflict_checks
        USING (tenant_id = current_setting('app.tenant_id')::UUID)
    """)
    op.execute("""
        CREATE POLICY tenant_billing_isolation ON billing_events
        USING (tenant_id = current_setting('app.tenant_id')::UUID)
    """)

    # ═══ TRIGGERS ═══════════════════════════════════════════

    op.execute("""
        CREATE OR REPLACE FUNCTION update_updated_at()
        RETURNS TRIGGER AS $$
        BEGIN NEW.updated_at = now(); RETURN NEW; END;
        $$ LANGUAGE plpgsql
    """)
    op.execute("""
        CREATE TRIGGER trg_docs_updated_at
        BEFORE UPDATE ON legal_documents
        FOR EACH ROW EXECUTE FUNCTION update_updated_at()
    """)

    # ═══ PHASE 1 JURISDICTION SEED DATA ═══════════════════

    op.execute("""
        INSERT INTO jurisdictions
            (country, level, name, name_fr, code, languages,
             coverage_status, phase, license_cleared)
        VALUES
            ('CA','federal','Canada (Federal)','Canada (fédéral)',
             'CA-FED', ARRAY['en','fr'], 'pending', 1, false),
            ('CA','province','Ontario','Ontario',
             'CA-ON', ARRAY['en'], 'pending', 1, false),
            ('CA','province','British Columbia','Colombie-Britannique',
             'CA-BC', ARRAY['en'], 'pending', 1, false),
            ('CA','province','Alberta','Alberta',
             'CA-AB', ARRAY['en'], 'pending', 1, false),
            ('US','federal','United States (Federal)',NULL,
             'US-FED', ARRAY['en'], 'pending', 2, true),
            ('US','state','California',NULL,
             'US-CA', ARRAY['en'], 'pending', 2, false),
            ('US','state','New York',NULL,
             'US-NY', ARRAY['en'], 'pending', 2, false),
            ('US','state','Texas',NULL,
             'US-TX', ARRAY['en'], 'pending', 2, false),
            ('US','state','Florida',NULL,
             'US-FL', ARRAY['en'], 'pending', 2, false),
            ('US','state','Washington',NULL,
             'US-WA', ARRAY['en'], 'pending', 2, false)
    """)


def downgrade() -> None:
```

```
    # Drop in reverse dependency order
    for table in [
        'agent_manifest','eval_runs','ingest_jobs','query_cache',
        'audit_log','conflict_checks','monitor_rules','legal_updates',
        'legal_citations','legal_chunks','legal_documents',
        'jurisdictions','billing_events','tenants'
    ]:
        op.execute(f"DROP TABLE IF EXISTS {table} CASCADE")
    op.execute("DROP EXTENSION IF EXISTS vector")
    op.execute("DROP EXTENSION IF EXISTS pgcrypto")
    op.execute("DROP EXTENSION IF EXISTS pg_trgm")
    op.execute("DROP FUNCTION IF EXISTS update_updated_at CASCADE")
```

AEU-0 PACKAGE STATUS

All five artifacts are now complete and production-ready.[1] [2] [3]

Artifact	File	Status	Blocker
Project manifest	`AGENTS.md`	✅ Complete	None — load into ContextKeeper
Data source registry	`LEGAL_DATA_SOURCES.md`	✅ Complete	B-1, B-2: CLO clearance outstanding
Privacy framework	`PIPEDA_COMPLIANCE.md`	✅ Complete	Privacy Counsel date + signature
Grounding contract	`GROUNDING_CONTRACT.md`	✅ Complete	None — enforce at gateway layer
Schema migrations	`migrations/0001_initial_schema.py`	✅ Complete	B-5, B-6: GCS bucket + Ollama model

The single human action that unblocks AEU-1: CLO signs `LEGAL_DATA_SOURCES.md` for the Phase 1 sources (Justice Laws Canada + CanLII partnership confirmation). Once B-1 and B-2 resolve, `HordeMaster` promotes to AEU-1 and the Foundation lane fires. Everything after that is Horde-executed.[4] [5]

Say the word and AEU-1 code begins.

⁂

1. architecture-patterns.md
2. tech-stack-guide.md
3. best-practices.md
4. https://stackconsulting.pro
5. https://www.devopsschool.com/blog/multi-agent-systems-engineer-role-blueprint-responsibilities-skills-kpis-and-career-path/